

Network Security Lab Report

Packet Sniffing, Spoofing & Traceroute

Lab 1 — Hands-On Exercises

Task 1: Packet Sniffing

1.1 Capturing All Packets Between User and Victim

The first exercise involves capturing and analysing all traffic on the network interface. The following sequence was observed:

- An ARP request was sent from the user to the network, asking for the MAC address of 10.9.0.5 (the victim).
- An ARP reply was received, mapping 10.9.0.5 to MAC address 02:42:0a:09:00:05.
- ICMP echo request/reply pairs (ping) then flowed between 10.9.0.6 (user) and 10.9.0.5 (victim) at one-second intervals.

What is ICMP?

ICMP (Internet Control Message Protocol) is a network-layer protocol used for error reporting, diagnostics, and network management. It allows devices to signal issues such as unreachable destinations or packet timeouts. Key uses include:

- ping — uses ICMP Echo Request / Reply to test host reachability.
- traceroute — maps network paths using ICMP Time Exceeded messages.

1.2 Applying an ICMP Filter

Applying a capture filter to show only ICMP traffic removes ARP noise and lets us focus solely on the ping exchange. The filtered output confirms the same request/reply pattern repeating every second between 10.9.0.6 and 10.9.0.5.

1.3 Shifting Focus to Port 23 (Telnet)

A Telnet session was established from the user machine to the victim:

```
root@b5c0594c1365 /# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^['.
```

```
hi
```

From the attacker VM, tcpdump was used to intercept traffic on port 23:

```
# Listen on any interface, port 23, from host 10.9.0.6
tcpdump -i any port 23 and host 10.9.0.6
```

The attacker VM also used netcat to receive a forwarded copy of the Telnet session:

```
root@180795fa262e /# nc -lnvp 23
Listening on 0.0.0.0 23
Connection received on 10.9.0.6 54484
    !''hi
hi
```

1.4 Listening by Subnet Mask

The fourth variant uses tcpdump with a subnet filter to capture all traffic within the 10.9.0.0/24 network:

```
tcpdump -i any net 10.9.0.0/24
```

This captures all ICMP echo request/reply exchanges between all hosts on the subnet, confirming comprehensive visibility at the network level.

Task 2: IP Spoofing

2.1 Objective

The goal is to send a spoofed ICMP echo request so that the victim believes the ping originated from the user (10.9.0.6) when it was in fact crafted by the attacker. This causes the victim to send its reply to the user's IP, even though the user never transmitted any request.

2.2 Spoofing Script

The following Python script using Scapy crafts the spoofed packet:

```
from scapy.all import *

# Spoofed ICMP echo request (ping)
```

```
# src is the USER's IP - attacker impersonates the user
packet = IP(src="10.9.0.6", dst="10.9.0.5") / ICMP()
send(packet, verbose=1)
```

2.3 Results

Three spoofing scenarios were tested:

Spoofed	Destination	Outcome
10.9.0.6 (User)	10.9.0.5 (Victim)	Victim replied to 10.9.0.6. User received the reply despite never sending a request.
10.9.0.7 (Invented)	10.9.0.5 (Victim)	Victim received the request but had no route to 10.9.0.7, so no reply was sent.
8.8.8.8 (Google)	10.9.0.5 (Victim)	Victim replied toward 8.8.8.8. The reply was routed externally and never reached the attacker.

Key observation: spoofing works when the forged source IP is reachable from the victim. When an invented or unroutable IP is used, the victim still receives the request but cannot deliver its reply.

Task 3: Custom Traceroute

3.1 How Traceroute Works

Traceroute discovers the path packets take to a destination by sending probe packets with incrementally increasing TTL (Time To Live) values. Each router along the path decrements the TTL; when it reaches zero the router discards the packet and sends back an ICMP Time Exceeded message, revealing its IP address.

3.2 Python Traceroute Script

The custom implementation uses Scapy to replicate this behaviour:

```
from scapy.all import IP, ICMP, srl
import sys

def traceroute(target, max_hops=30):
    print(f"Tracing route to {target}...\n")
```

```
for ttl in range(1, max_hops + 1):
    # Build an IP/ICMP packet with increasing TTL
    pkt = IP(dst=target, ttl=ttl) / ICMP()

    # Send and wait for a reply
    reply = sr1(pkt, timeout=2, verbose=False)

    if reply is None:
        print(f"{ttl}: * * * (No response)")
    else:
        print(f"{ttl}: {reply.src}")
        if reply.src == target:
            print("Trace complete.")
            break

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Usage: python3 traceroute.py <target>")
        sys.exit(1)
    target = sys.argv[1]
    traceroute(target)
```

3.3 Trace to Google DNS (8.8.8.8)

Running the custom script against 8.8.8.8 produced the following 11-hop path:

```
python3 traceroute.py 8.8.8.8
```

```
Tracing route to 8.8.8.8...
```

```
1: 10.4.68.1
2: 10.253.2.101
3: 10.0.66.2
4: 213.248.70.176
5: 62.115.136.194
6: 62.115.136.146
7: * * * (No response)
8: 62.115.151.27
9: 108.170.236.173
10: 142.250.214.193
11: 8.8.8.8
Trace complete.
```

3.4 Comparison with Built-in traceroute

The system traceroute command was also run against the same target. While the overall path to 8.8.8.8 was the same, minor differences were observed, including:

- Slightly different intermediate hop IPs at certain points, reflecting load-balanced routing.
- The built-in traceroute sends three probes per hop and displays latency (ms) for each; the custom script sends one probe and shows only the IP.
- Hop 7 showed no response in the custom script ('* * *') while the built-in tool revealed partial timing data, suggesting firewall filtering of certain ICMP types.

Summary of Differences

The custom Python/Scapy implementation faithfully reproduces the core traceroute mechanism. Divergences arise from probe count (1 vs 3 per hop), ICMP type handling differences, and load-balancing in the upstream network that can route individual packets differently.